

NetworkX & PyTorch Geometric

def environment Python for dev → **uv** package and project manager for Python written in Rust
 . 10-100x faster than pip
 with universal lockfile

how to install for macOS/Linux Windows } CLI script to download and run

release uv.lock } create folder of the project → in the released project you can run Jupyter Lab
 pyproject.toml } uv sync
 uv run jupyter lab → open automatically a browser instance
 torch, torch-geometric, networkx
 ↓
 PyG → if you need extra packages (see "Installation" on the website)
 uv add torch-cluster
 torch-scatter
 torch-sparse

PyG is library built on PyTorch to implement and train Graph Neural Network

for DL on graphs or irregular structures + mini-batch data loader for large graphs support multi-GPU

homogeneous graph: torch-geometric.data.Data

- x = node features $\mathbb{R}^{m \times d_m}$
- edge_index = graph adjacency matrix in COO format
- edge_attr = edge features $\mathbb{R}^{e \times d_e}$
- y = Target(s)
- pos = node coordinates $\mathbb{R}^{m \times 2}$

CONVENTIONAL NOT MANDATORY
 +
 additional fields: masks
 metadata

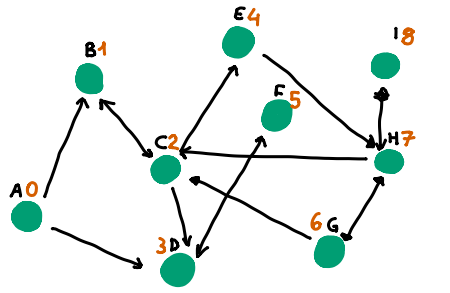
minimal representation: **edge-index** → COO a) first row contains index of source nodes
 b) second row " " " " " target nodes

for undirected graphs, PyG stores both directions, i.e. $A - B \Leftrightarrow A \rightarrow B$
 $B \rightarrow A$

x : feature

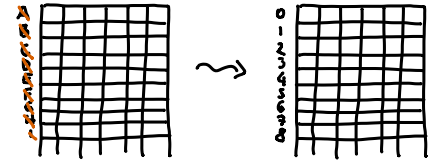
$\downarrow$
 torch.tensor([
 [s₀, ..., s_E],
 [t₀, ..., t_E],
], dtype=torch.long)
 integer

torch.tensor([
 [f₀],
 ...,
 [f_m],
])



we cannot use nodes' label or ids directly but ONLY INTEGERS as index = it is NECESSARY that elements in edge_index hold indices in the range [0, m-1]
 (g.validate(raise_on_error=True))

? : facilitate look-up in the edge and node feature matrices



minimal info for node classification

graph = Data(
 x =
 edge_index =
 y =

Labels for nodes ⇒ accessible by node index

e.g. $y[2] \approx$ label of node "D"

load a benchmark dataset "Cora" available through Planetoid

torch-geometric.dataset.Planetoid

Planetoid(root="data/planetoid", name="Cora")

for Cora we have 3 masking vectors at node-level

- train_mask
- val_mask
- test_mask

? which type train_mask.dtype
 ? how many for set train_mask.sum().item()
 ? how to use them → selection x[train_mask]

→ basic info

order g.num_nodes
 size g.num_edges
 shape feature g.x.shape | g.num_node_features
 edge_index g.edge_index.shape
 target shape g.y.shape
 # classes dataset.num_classes | g.y.unique()

graph object ≠ dataset object

collection of independent graphs
 Dataset / InMemoryDataset

graph data loaded on demand

if dataset do not provide masks, we use torch-geometric.transforms.RandomNodeSplit

- add mask attributes to Data/HeteroData
- split: 'train-rest', 'test-rest', 'random' → $g = \text{split}(g)$
- num_splits
- num_train_per_class
- num_val
- num_test
- Key

multilayer networks modelled by torch-geometric.data.HeteroData

store node and edge attributes and layers for each node type → each node type has feature matrix $e \in \mathbb{R}^{1 \times d}$

relation type

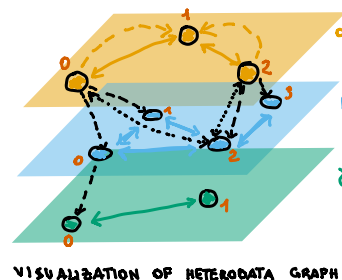
identified by triplet (source_node_type ≈ α , relation_type, target_node_type ≈ β)

≈ $B_{\alpha, \beta}$

each relation type use edge_index ⇒ SAME CONSTRAINTS ON INDICES:
 $\forall d \in [0, d_d - 1]$

marks associated with node and relation type

- basic info g.node_types, g.edge_types, g.metadata()
 g.x_dict, g.edge_index_dict



VISUALIZATION OF HETEROGRAPH

different type of relations for inter link and for intra link

NetworkX most used library for graph analysis in Python, more flexible than PyG

hashable Python obj as nodes + Python obj as edge attributes

main: Graph, DiGraph, MultiDiGraph, MultiGraph
 ↓
 undirected graph

add_node() + other attributes: feature vector, label, metadata → not separate but each node has its own data

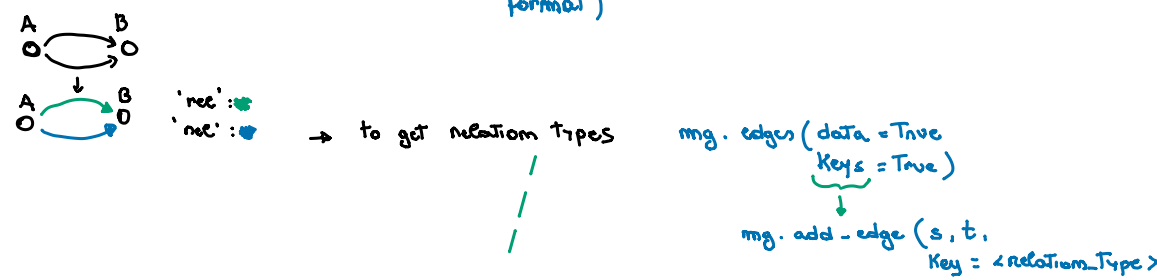
Key-valued dict Key = node id (hashable obj)

add_edge(s, t, ...) → g.edges(data=True)

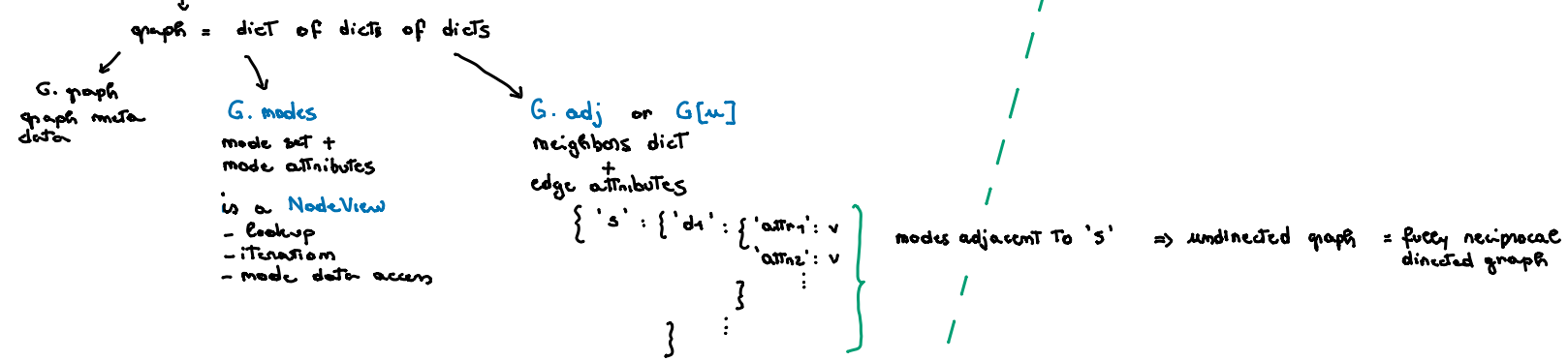
• build node feature matrix `mp.stack(<list of vectors>)`
`torch.tensor(↓, dtype)`

• from graph to Scipy sparse matrix
`mx.to_scipy_sparse_array(G, modelist format)`

in **Multigraph** put the "relation type" as attribute to distinguish



• mx uses **adjacency structures** based on dictionary → sparse graphs + arbitrary node and edge features

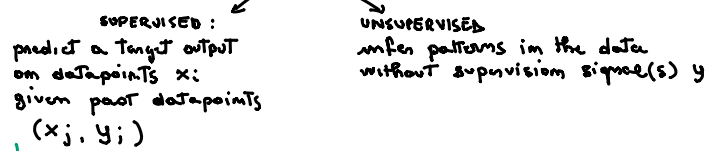


DiGraph has two more structures `G.succ = G.adj`
`G.pred`

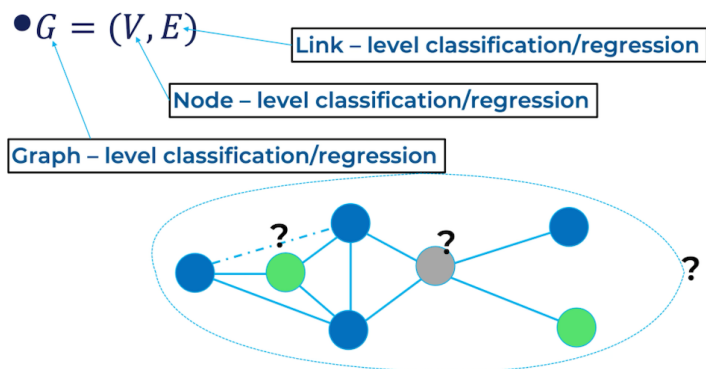
Multigraph extends the nested structure of adjacency dict: node → neighbors → **edge key** → edge attributes
relation type

→ **from NetworkX to PyG** → Graph, DiGraph → `torch_geometric.utils.from_networkx(Graph)` → Data object
↓
Mult(Di) Graph → script for conversion
[see code repository → `networkx_to_pytorchdata()`]

Since ML is problem-driven ML models are categorized by the task they solve



in ML on graphs these categories are less informative $\rightarrow$ boundaries among traditional tasks are blurry



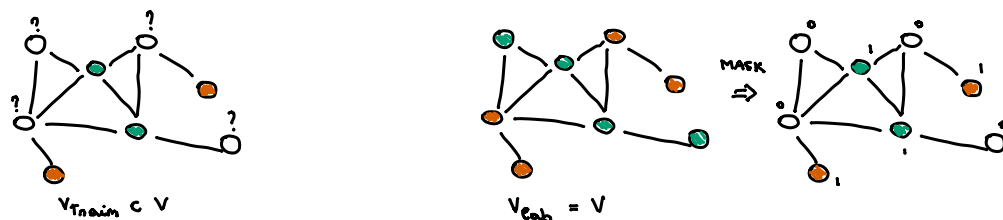
graph made by different component $\Rightarrow$ each component often labelled (all or partially!) $\Rightarrow$ each component is a subject of supervised learning task

NODE-LEVEL TASKS

NODE CLASSIFICATION / REGRESSION: goal is to predict the label $y_u \in \mathbb{R}$ or $\mathbb{C}$, $u \in V(G)$ when only a subset of nodes in G , i.e. V_{train} , has labelled nodes $\in G$ (set of nodes)

input: $G=(V, E)$ connection among nodes in V
 $V_{\text{train}} \subset V = \{u \in V \mid \exists y \in S, \text{label}(u)=y\} \Rightarrow$ labelled nodes $\Rightarrow f: V \rightarrow S$ $S=\mathbb{R}$
 optional $\forall u \in V, x_u \in \mathbb{R}^{d_m} \rightarrow$ node feature vector $S=\mathbb{C}$

we start from C (proper subset) $\Rightarrow$ in the original graph a few nodes haven't labels
 $= (V_{\text{labelled}} = V) \Rightarrow$ " " " all nodes have labels $\rightarrow$ we have to force $V_{\text{train}} \subset V$ by masking V



examples: protein functions
 topic of documents
 bot detection

$\bullet$ G may be disconnected $\rightarrow f$ is applied on nodes in another connected component

node classification/regression is not a standard supervised task: nodes and links are not INDEPENDENT: $D = \{(x_u, y_u)\}_{u=1}^{|V|}$

AND

IDENTICALLY DISTRIBUTED

$\forall u \in V (x_u, y_u) \sim P_{xy}$
 $\downarrow$
 all instances are generated by the same distribution P_{xy}

nodes have different structural roles $\rightarrow$ features and labels depends on structural properties determine the roles

in multilayers net
 $P_{xy}^A(x^A, y^A) \neq P_{xy}^B(x^B, y^B)$

$\bullet$ HETEROPHILY VS

- $\bullet$ in citation networks two papers connected are more likely to belong to the same research area
- $\bullet$ HOMOPHILY: tendency of nodes to share attributes similar with their neighbors

- $\bullet$ STRUCTURAL EQUIVALENCE: nodes with similar local structure even if in far region of the graph behave similarly

- $\bullet$ in sensor network, a value of a sensor will be close the values detected by nearby sensors

$$P((x_1, y_1) \dots (x_m, y_m)) = \prod_{i=1}^m P_{xy}(x_i, y_i)$$

if we know (x_u, y_u) we do not add additional information on (x_v, y_v) $u \neq v$

$\downarrow$
 each instance is independent from the others

BUT

in graph ML we have a relational structure $G=(V, E)$

links introduce STATISTICAL DEPENDENCY

$P(V|X, G)$

y_u depends on x_u and all the labels and features of nodes v adjacent to u

$\downarrow$

WE LEVERAGE THESE DEPENDENCIES !!

RELATION / LINK-LEVEL TASK

- $\bullet$ cases where the relationship info is missing $\Rightarrow$ link prediction, graph completion, relational inference, relation prediction

$\downarrow$

most popular task in graph ML because used for recommender systems predict drug side effects new facts inference in KG

$\bullet$ SETTING: given V and $E_{\text{train}} \subset E$ incomplete set of links

$\rightarrow$ GOAL: infer $e \in E/E_{\text{train}}$

complexity depends on the type of graph $\rightarrow$ social networks $\Rightarrow$ heuristic based on social theories

biological knowledge graph requires more complex reasoning

$\downarrow$
 determine the inductive bias that is domain specific

formulation depends on the availability of temporal info

STATIC NETWORK

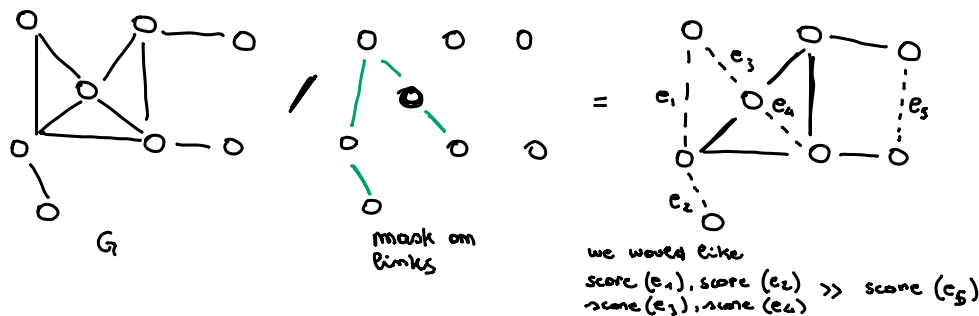
E_{train} built by removing links at random from E and predict them

TEMPORAL NETWORK

given $G[t_0, t_0']$ the network slice from $t_0 \rightarrow t_0'$ as E_{train} , predict links not in E_{train} that appear in $G[t_1, t_1']$ s.t. $t_1' > t_1 \geq t_0' > t_0$



many times is predicting/assigning a score/probability of being in $G[t_1, t_1']$

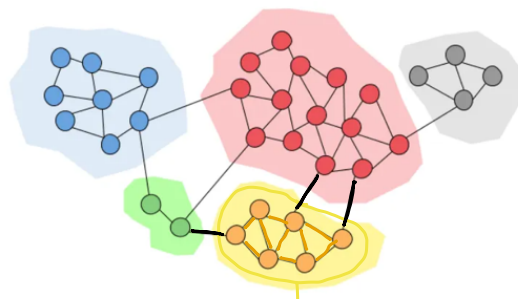


CLUSTERING / COMMUNITY DETECTION

analogues of unsupervised clustering = identify nodes segregated in clusters

↓
 info / label about community membership is missing,
 only use $G=(V, E)$

community structure where nodes are more likely to form links with other nodes in the same community



links between nodes in the community $\gg$ # links connecting nodes in the community to nodes outside it (3)

GRAPH -LEVEL TASKS

classification, regression, clustering over a set of graphs

↓
 given a molecule (graph) predict toxicity or solubility
 identify if a program (syntax and data flow graph) is malicious

analogues of standard supervised tasks since iid assumption is valid $\Rightarrow$ in $\mathcal{D} = \{(G_i, y_i)\}_{i=1}^g$ each instance is a graph and G_i are i.i.d.

use a similarity score between pairs of graph